

COMSATS University Islamabad

Attock Campus



Department Of Computer Science

<i>Course:</i>	<i>Information Security</i>
<i>Instructor:</i>	<i>Mrs.Ambreen Gul</i>
<i>Due Date:</i>	<i>19th May, 2026</i>
<i>SEMESTER PROJECT</i>	

Student Details

<i>Registration No:</i>	<i>Name</i>
<i>FA24-BSE-025</i>	<i>Manahil Kamran</i>
<i>FA24-BSE-030</i>	<i>Sayed Aliza Hashmi</i>

Contents

Block Chain Technology	3
Code:	3
Introduction.....	6
Objectives.....	6
Code Implementation (Python).....	6
Class: Block	6
Class: Blockchain.....	6
How the System Works	6
Blockchain Initialization	6
Adding Transactions	6
Mining.....	6
Validation.....	7
Tampering Test	7
Real-World Scenario: Bitcoin	7
How it works:	7
Relation to our code:.....	7
Advantages & Disadvantages:.....	7
Applications Beyond Bitcoin.....	8
Conclusion	8
Output:	8

Block Chain Technology:

Code:

```
import hashlib, json, time

class Block:
    def __init__(self, index, transactions, previous_hash):
        self.index = index
        self.timestamp = time.time()
        self.transactions = transactions
        self.previous_hash = previous_hash
        self.nonce = 0
        self.hash = self.calc_hash()

    def calc_hash(self):
        block_string = json.dumps({
            'index': self.index,
            'timestamp': self.timestamp,
            'transactions': self.transactions,
            'previous_hash': self.previous_hash,
            'nonce': self.nonce
        }, sort_keys=True)
        return hashlib.sha256(block_string.encode()).hexdigest()

    def mine(self, difficulty):
        print(f"Mining Block #{self.index}...")
        while self.hash[:difficulty] != '0'*difficulty:
            self.nonce += 1
            self.hash = self.calc_hash()
        print(f"Mined! Nonce: {self.nonce}, Hash: {self.hash[:10]}...")

class Blockchain:
    def __init__(self, difficulty=3):
        self.chain = []
        self.pending_tx = []
        self.difficulty = difficulty
        self.create_genesis()

    def create_genesis(self):
        genesis = Block(0, [], "0")
        genesis.mine(self.difficulty)
        self.chain.append(genesis)

    def add_tx(self, sender, receiver, amount):
        self.pending_tx.append({
            'sender': sender,
            'receiver': receiver,
            'amount': amount,
```

```

        'timestamp': time.time()
    })
    print(f"Added: {sender} → {receiver}: ${amount}")

def mine_pending(self, miner):
    if not self.pending_tx:
        print("No transactions to mine")
        return

    block = Block(
        len(self.chain),
        self.pending_tx.copy(),
        self.chain[-1].hash
    )
    block.mine(self.difficulty)
    self.chain.append(block)

    # Clear pending, add reward
    self.pending_tx = []
    self.add_tx("Network", miner, 1.0)
    print(f"Block #{block.index} added to chain!")

def is_valid(self):
    print("\nValidating chain...")
    for i in range(1, len(self.chain)):
        current = self.chain[i]
        prev = self.chain[i-1]

        # Check hash and PoW
        if current.hash != current.calc_hash():
            print(f"✗Block #{i} hash invalid")
            return False
        if current.hash[:self.difficulty] != '0'*self.difficulty:
            print(f"✗Block #{i} PoW invalid")
            return False
        if current.previous_hash != prev.hash:
            print(f"✗Block #{i} chain broken")
            return False

    print("✓Blockchain valid!")
    return True

def show(self):
    print("\n" + "="*50)
    print("BLOCKCHAIN")
    print("="*50)
    for block in self.chain:
        print(f"\nBlock #{block.index}:")

```

```

print(f" Hash: {block.hash[:10]}...")
print(f" Prev: {block.previous_hash[:10]}...")
print(f" Nonce: {block.nonce}")
print(f" Txns: {len(block.transactions)}")

# ===== DEMO =====
print("="*50)
print("SIMPLIFIED BLOCKCHAIN")
print("="*50)

# Create blockchain
bc = Blockchain(difficulty=3)

# Add transactions
bc.add_tx("Alice", "Bob", 50)
bc.add_tx("Bob", "Charlie", 25)

# Mine block
bc.mine_pending("Miner1")

# Add more
bc.add_tx("Charlie", "David", 10)
bc.add_tx("David", "Eve", 5)

# Mine another
bc.mine_pending("Miner2")

# Show chain
bc.show()

# Validate
bc.is_valid()

# Try to tamper
print("\n" + "="*50)
print("TAMPERING TEST")
print("="*50)
print("Tampering with Block #1...")
bc.chain[1].transactions[0]['amount'] = 999
bc.chain[1].hash = bc.chain[1].calc_hash() # Recalc but don't re-mine

# Show validation fails
bc.is_valid()

```

Introduction

Blockchain is a distributed ledger technology that ensures secure, tamper-resistant storage of data. Bitcoin, the first and most widely used cryptocurrency, is built entirely on blockchain principles. This project demonstrates a simplified blockchain in Python, focusing on SHA-256 hashing and Proof-of-Work mining, the same cryptographic techniques used in Bitcoin.

Objectives

- Implement a mini blockchain using Python.
- Demonstrate block creation, hashing, mining, and validation.
- Show how tampering is detected.
- Relate the simplified model to Bitcoin's real-world implementation.
- Discuss advantages and disadvantages of blockchain in Bitcoin.

Code Implementation (Python)

The project is implemented in Python with two main classes:

Class: Block

- Stores block index, timestamp, transactions, previous hash, and nonce.
- Uses SHA-256 hashing to generate a secure fingerprint.
- Mining function performs Proof-of-Work by finding a hash with leading zeros.

Class: Blockchain

- Initializes the chain with a genesis block.
- Adds transactions to a pending list.
- Mines pending transactions into new blocks.
- Validates the chain by checking hashes, Proof-of-Work, and links.
- Detects tampering when any transaction is altered.

How the System Works

Blockchain Initialization – Genesis block is created and mined.

Adding Transactions – Pending transactions are stored until mined.

Mining – Proof-of-Work ensures computational difficulty.

Validation – Each block's hash and link are checked.

Tampering Test – Any change invalidates the chain.

Real-World Scenario: Bitcoin

Where used: Peer-to-peer digital currency system.

How it works:

- Transactions (Alice → Bob) are grouped into blocks.
- Miners solve Proof-of-Work puzzles using SHA-256.
- Valid blocks are added to the chain.
- Miners receive Bitcoin rewards.

Relation to our code:

- Pending transactions → Bitcoin mempool.
- Mining function → Bitcoin Proof-of-Work.
- `is_valid()` → Bitcoin's consensus validation.
- Tampering test → immutability of Bitcoin ledger.

Advantages & Disadvantages:

Advantages:

- Decentralized, no central authority.
- Secure and tamper-proof via SHA-256.
- Transparent public ledger.
- Immutable records.

Disadvantages:

- High energy consumption due to mining.
- Slow transaction speed compared to centralized systems.
- Price volatility makes it risky as currency.
- Scalability limits transactions per second.

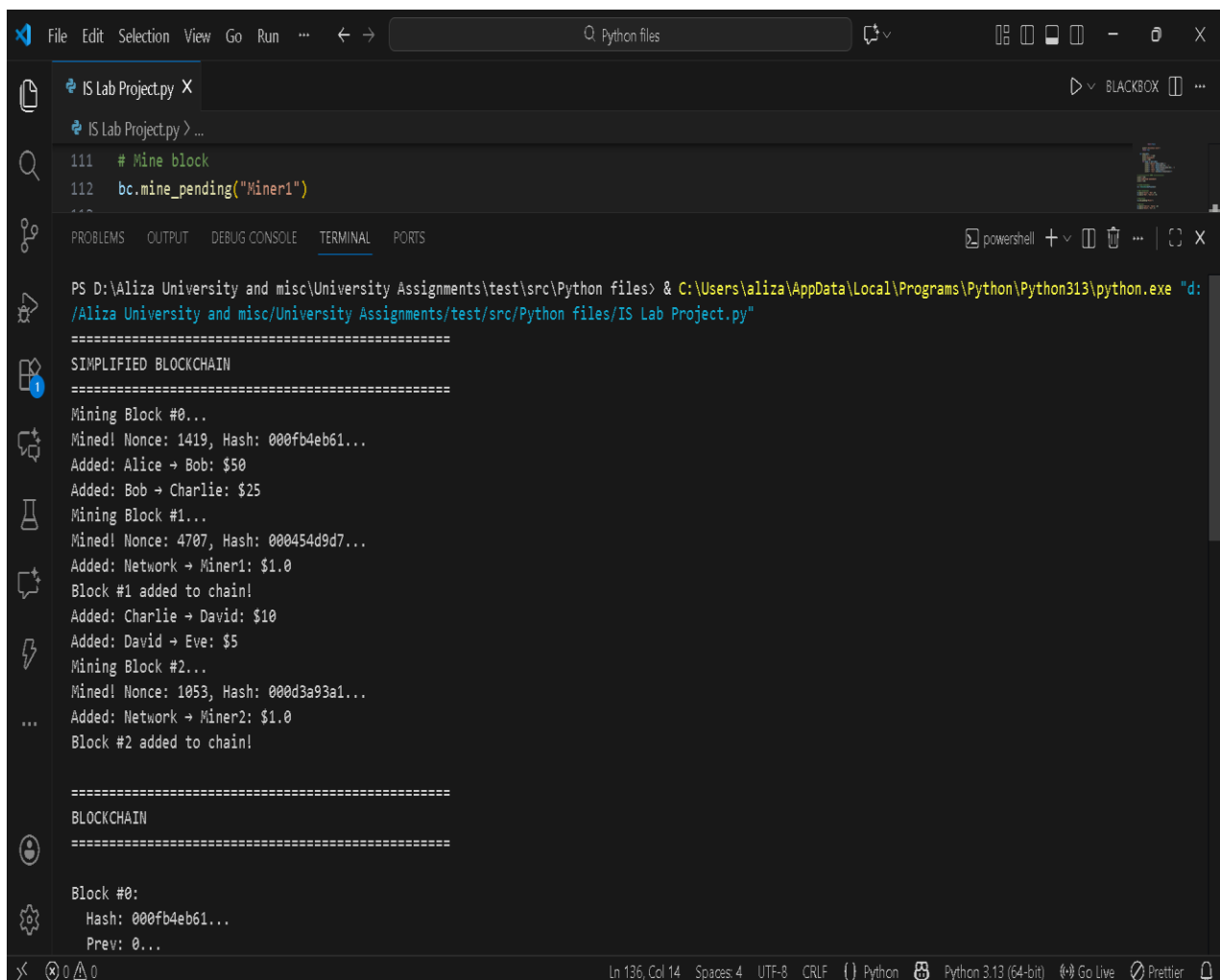
Applications Beyond Bitcoin

- Supply Chain Tracking – Immutable product history.
- University Certificates – Secure digital diplomas.
- Medical Records – Tamper-proof patient data.
- Voting Systems – Transparent, fraud-resistant elections.

Conclusion

This project demonstrates the core principles of blockchain using Python and relates them directly to Bitcoin. By implementing hashing, Proof-of-Work, mining, validation, and tampering detection, we gain a clear understanding of how Bitcoin maintains trust, security, and immutability in a decentralized environment.

Output:



```
File Edit Selection View Go Run ... Python files
IS Lab Project.py X
IS Lab Project.py > ...
111 # Mine block
112 bc.mine_pending("Miner1")
...
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\Aliza University and misc\University Assignments\test\src\Python files> & C:\Users\aliza\AppData\Local\Programs\Python\Python313\python.exe "d:/Aliza University and misc/University Assignments/test/src/Python files/IS Lab Project.py"
=====
SIMPLIFIED BLOCKCHAIN
=====
Mining Block #0...
Mined! Nonce: 1419, Hash: 000fb4eb61...
Added: Alice → Bob: $50
Added: Bob → Charlie: $25
Mining Block #1...
Mined! Nonce: 4707, Hash: 000454d9d7...
Added: Network → Miner1: $1.0
Block #1 added to chain!
Added: Charlie → David: $10
Added: David → Eve: $5
Mining Block #2...
Mined! Nonce: 1053, Hash: 000d3a93a1...
Added: Network → Miner2: $1.0
Block #2 added to chain!

=====
BLOCKCHAIN
=====

Block #0:
Hash: 000fb4eb61...
Prev: 0...
Ln 136, Col 14 Spaces: 4 UTF-8 CRLF Python Python 3.13 (64-bit) Go Live Prettier
```



```
File Edit Selection View Go Run ... Python files
IS Lab Project.py X
IS Lab Project.py > ...
111 # Mine block
112 bc.mine_pending("Miner1")
...

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
powershell + - | | X

=====
BLOCKCHAIN
=====

Block #0:
  Hash: 000fb4eb61...
  Prev: 0...
  Nonce: 1419
  Txns: 0

Block #1:
  Hash: 000454d9d7...
  Prev: 000fb4eb61...
  Nonce: 4707
  Txns: 2

Block #2:
  Hash: 000d3a93a1...
  Prev: 000454d9d7...
  Nonce: 1053
  Txns: 3

Validating chain...
[✓] Blockchain valid!

=====
TAMPERING TEST
=====

Validating chain...
[✓] Blockchain valid!

=====
TAMPERING TEST
=====

Tampering with Block #1...

Validating chain...
[✗] Block #1 PoW invalid
PS D:\Aliza University and misc\University Assignments\test\src\Python files>
```